

COM106: Introduction to Databases

Entity-Relationship Modelling (Part 2 / 2)



COM106 - Introduction to Databases (S1 2627): Weekly Schedule

COM106 Weekly Schedule - Lecture & Lab Delivery

Week 9 Lab is the in-class exam (Set Exercise, 90-minute class test, 40% of module mark).

Week	Lecture	Lab / Seminar
1	Module Introduction & DB Basics and Security	Lab 1: Getting Started with MYSQL
2	Introduction to Entity-Relationship Modelling	Lab 2: Creating ER Diagrams with draw.io
3	ER Modelling 2 & Mapping to the Relational Model - Worked examples: attributes & data types - Connection traps: fan & chasm - Multiplicities (min, max) notation - Mapping rules: key copying & link tables - Completing the relational schema	Lab 3: ER Diagram Design from Scenarios - The ER modelling method - University Academic Management System - Library Management System - Hospital Management System - Online Course Platform

<https://github.com/KodZone/COM106-MYSQL>

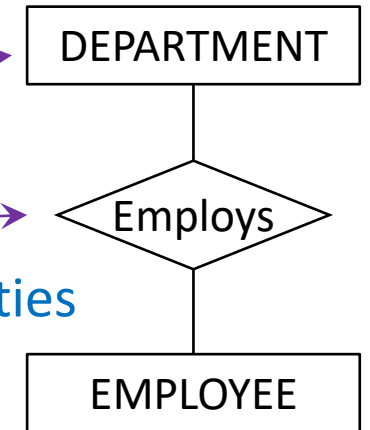


Recap

- **Entity-Relationship (ER) modelling** is a foundational method in **database design**, in which **entity** is a unique identity of object, concept, or thing in the real world that described by **attributes** and connected to other entities by **Relationship**.

- **Entity Relational (ER) Diagrams:**

- ENTITY TYPE: Represented by labelled rectangular box
- RELATIONSHIP: Represented by labelled diamond box between pair of entities



- **In ER Modelling, need to identify the following:**
 - Degree of a Relationship (unary, binary, ternary, etc)
 - Cardinality Ratio (1:1, 1:N, N:1, M:N)
 - Participation Constraints (Full or Partial)

Worked examples: attributes & data types

Studies Advice System

(Entities – Relationships - Degree of a Relationship - Cardinality
Ratio - Participation Constraints – Attributes)



Studies Advice System: 1) Entities

Draw an E-R Diagram which models the **University's Studies Advice System** (ie. *Individual Meetings between Student & Advisor*)

STUDENTS on a **COURSE** are allocated a **STUDIES ADVISER** who sets up **MEETINGS**
(Studies Advice Meetings with Students)

Timescale: One Academic Year

Solution

First, identify the Entities.

COURSE

MEETING

STUDENT

ADVISOR



Studies Advice System: 2) Relationships

STUDENTS on a **COURSE** are allocated a **STUDIES ADVISER** who sets up **MEETINGS** (Studies Advice Meetings with Students)

Timescale: One Academic Year

Solution

First, identify the Entities.

Then consider which relationships should be modelled?

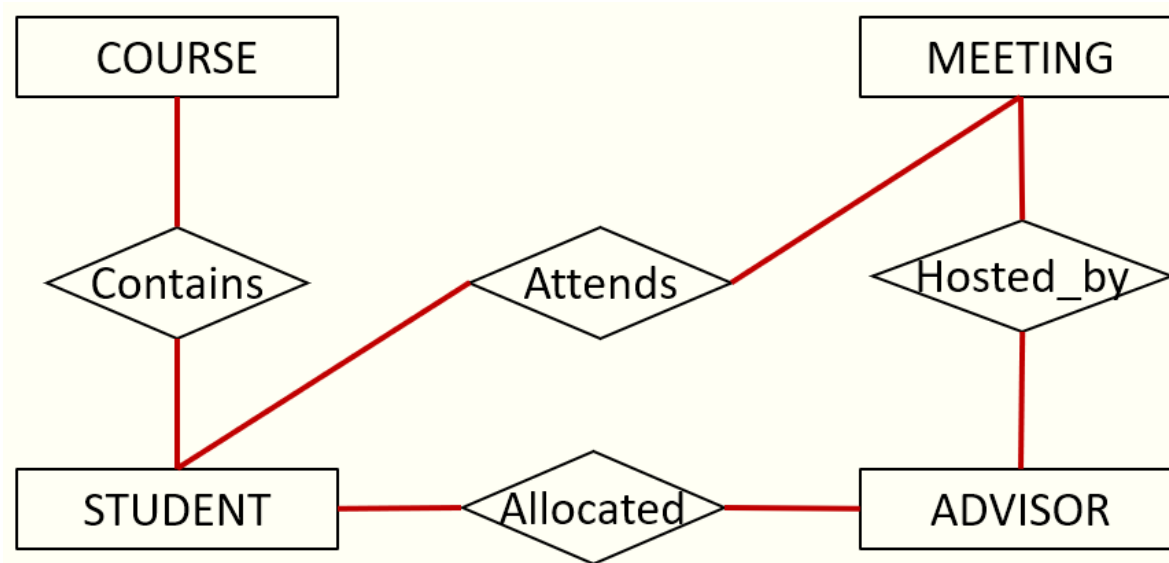
Most Important:

COURSE **Contains** **STUDENT**

STUDENT **Allocated** **ADVISOR**

STUDENT **Attends** **MEETING**

MEETING **Hosted By** **ADVISOR**





Studies Advice System: 3) Cardinality Ratio

Next, consider the Cardinality Ratio of each relationship

1:N

COURSE Contains STUDENT

Each **COURSE** contains more than one **STUDENT**

Each **STUDENT** is on just one **COURSE**

STUDENT Allocated ADVISOR

Each **STUDENT** is allocated one **ADVISOR**

Each **ADVISOR** has more than one **STUDENT** allocated

STUDENT Attends MEETING

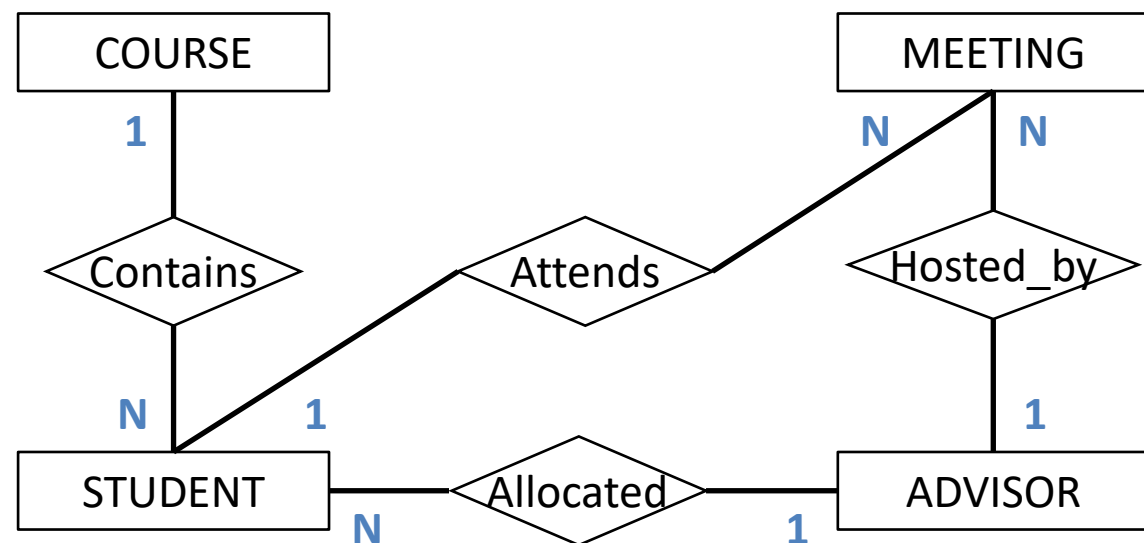
Each **STUDENT** attends more than one **MEETING**

Each **MEETING** is attended by one **STUDENT**

MEETING Hosted_by ADVISOR

Each **MEETING** is hosted by one **ADVISOR**

Each **ADVISOR** hosts more than one **MEETING**



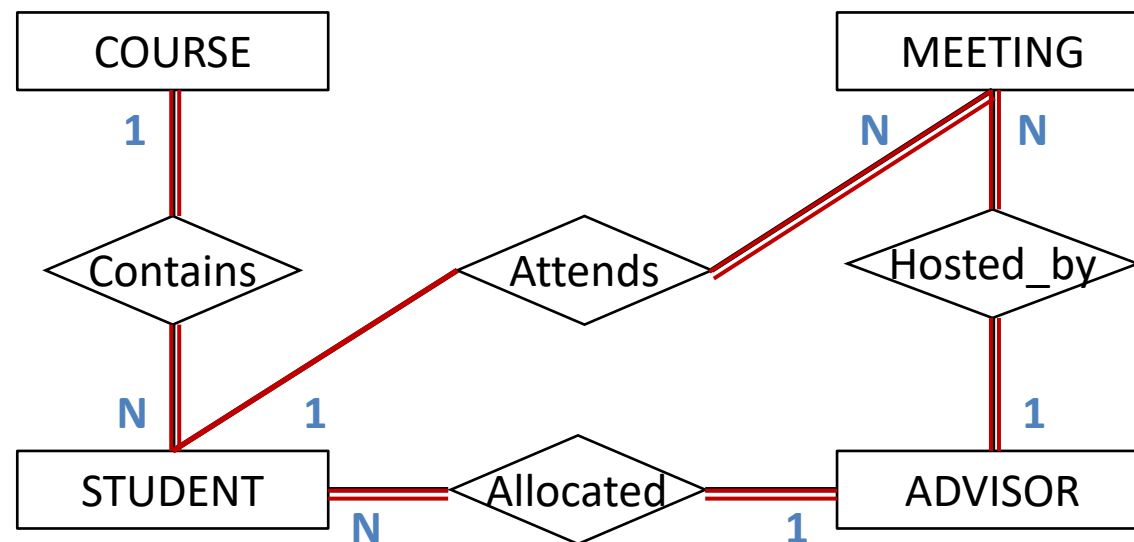


Studies Advice System: 4) Participation Constraints

Then, consider the Participation Constraints:

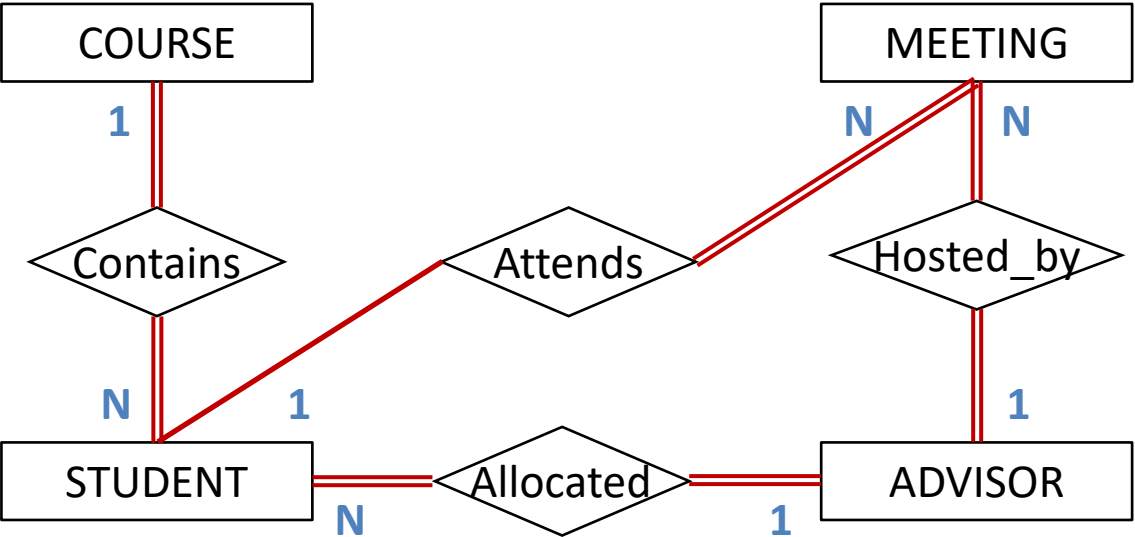
FULL if every instance of an entity type is linked to an instance of the other entity type in the relationship

PARTIAL if some instances are not linked





Studies Advice System





Studies Advice System: 5) Attributes

Consider the *potential attributes* of each entity type – and their datatypes

Just large enough for the largest possible value

COURSE

Potential Attributes	Datatype
course_code (Identifier)	char(6)
course_name	char(20)/varchar(20)
duration	tinyint
Course_director	char(20)

ADVISER

Potential Attributes	Datatype
advisor_code (Identifier)	char(9)
advisor_name	char(20)
room	char(5)
advisor_email	char(20)

STUDENT

Potential Attributes	Datatype
student_no (Identifier)	char(9)
student_name	(var)char(20)
gender	char(1)
date_of_birth	date
address	(var)char(30)
student_email	(var)char(20)

MEETING

Potential Attributes	Datatype
meeting_id (Identifier)	tinyint
meeting_number	tinyint (?)
date	date
time	Time(0)
place	(var)char(20)
details	(var)char(50)

See Microsoft MySQL Language Reference

Worked examples: attributes & data types

Olympic Games

(Entities – Relationships - Degree of a Relationship - Cardinality
Ratio - Participation Constraints – Attributes)



Olympic Games: 1) Entities

Draw an E-R Diagram which models an **Olympic Games**, where competitors from different countries compete in events held in various venues

Timescale: One Olympic games

1) Identify the **Entity Types**

Competitors from different countries
compete in events held in various venues

COUNTRY

VENUE

COMPETITOR

EVENT

By convention, Entity
Types are **SINGULAR**



Olympic Games: 2) Relationships

Draw an E-R Diagram which models an **Olympic Games** where ..
Competitors from different countries compete in events held in various venues

2) Identify all **Relationships** between **Entity Types**

The **most important** relationships which should be modeled include ...

COUNTRY Sends COMPETITOR

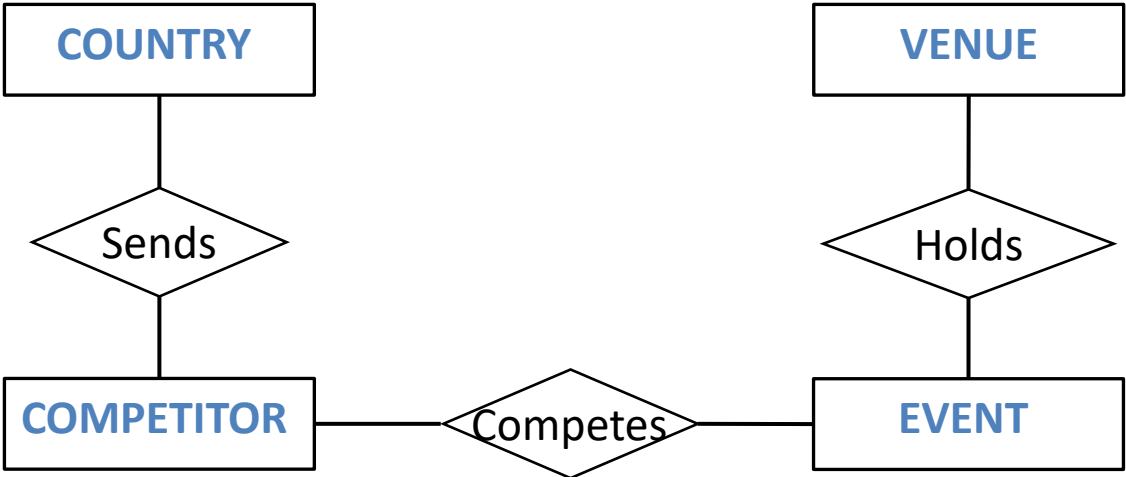
COMPETITOR Competes in EVENT

VENUE Holds EVENT

There are other possible relationships - should they be included or not?

COMPETITOR Attends VENUE ?

COUNTRY Participates_in EVENT ?





Olympic Games: 3) Cardinality Ratio

Draw an E-R Diagram which models an **Olympic Games** where ..
Competitors from different countries compete in events held in various venues

3) Determine the **Cardinality Ratios** of each relationship

Sends 1:N

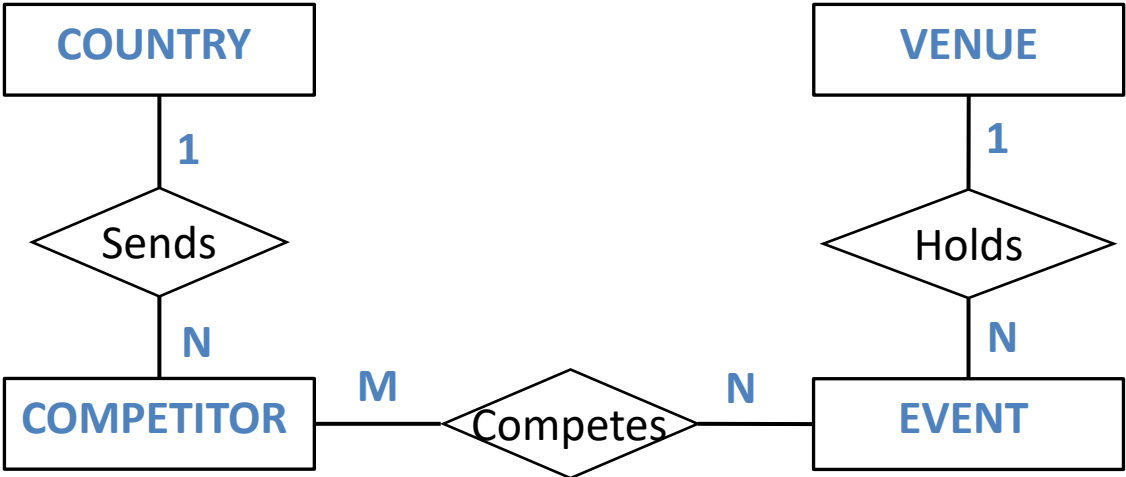
*A country can send more than one competitor
A competitor must be sent by one single country*

Competes M:N

*A competitor can compete in more than one event
An event has more than one competitor*

Holds 1:N

*A venue can hold more than one event
Each event is held in only one venue*



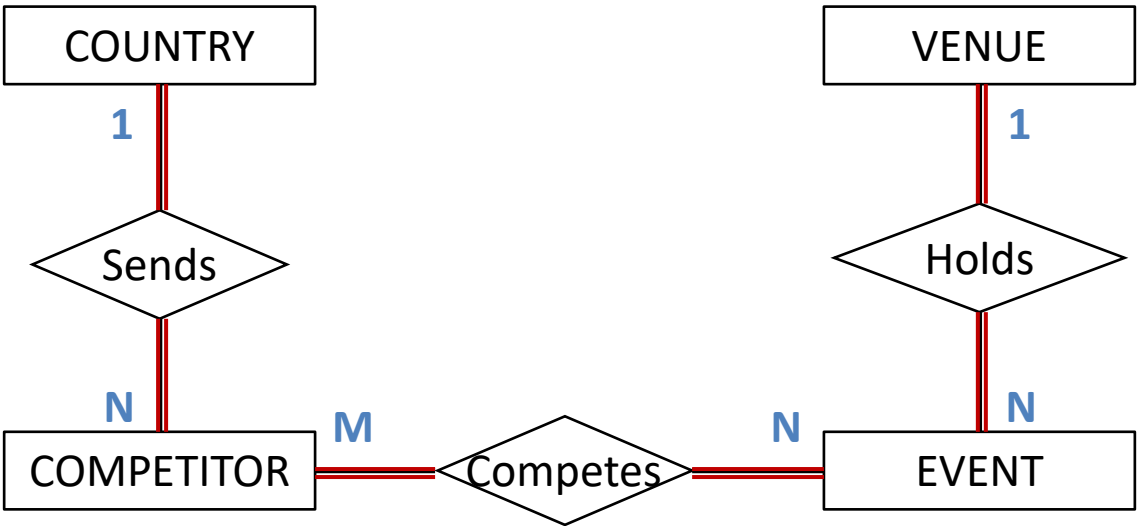


Olympic Games: 4) Participation Constraints

Draw an E-R Diagram which models an **Olympic Games** where ..
Competitors from different countries compete in events held in various venues

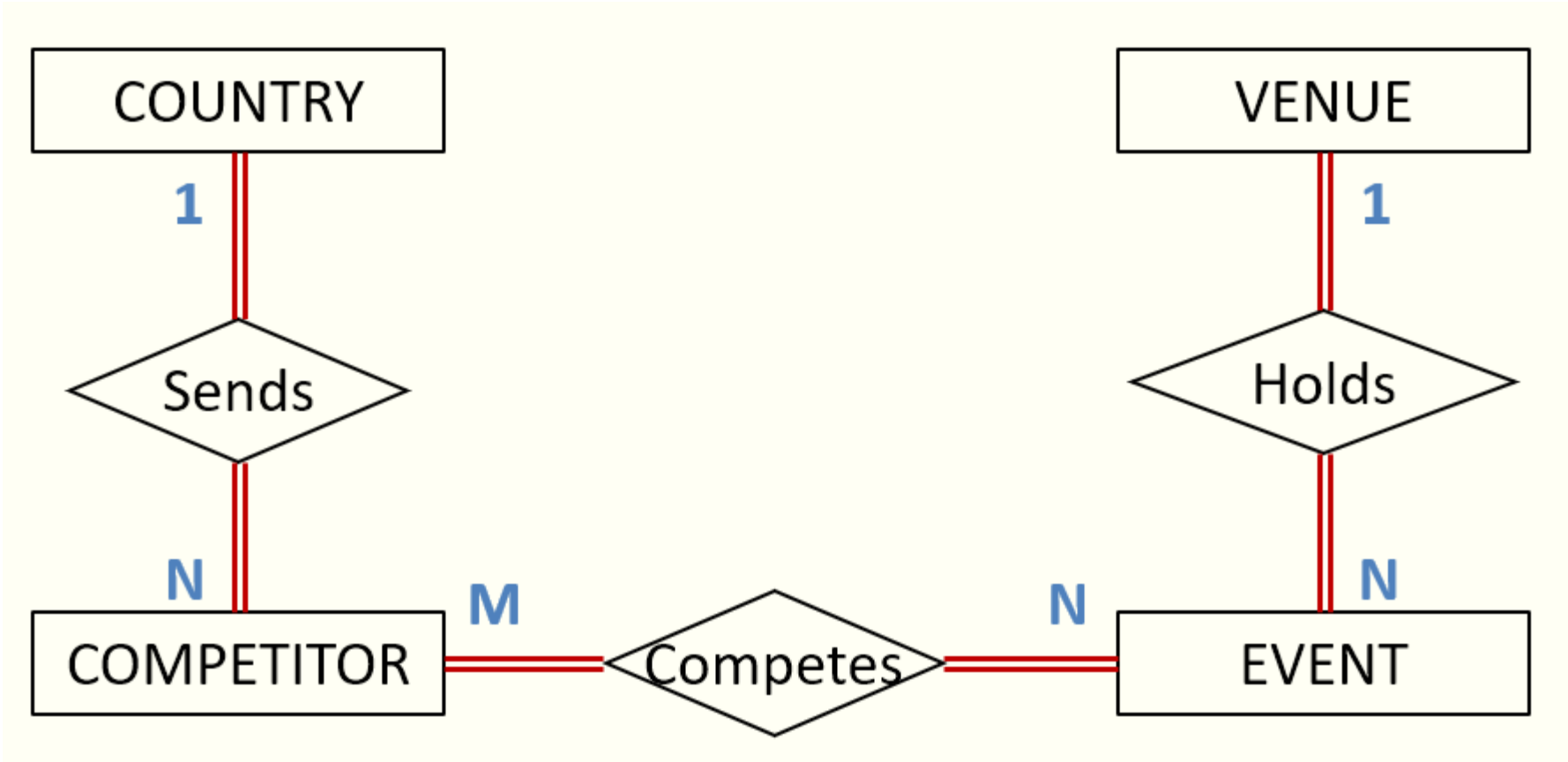
4) Identify the **Participation Constraints** of each entity type in each relationship

- COUNTRY** in **Sends** - **FULL** (if a country has no competitors they wouldn't be there!)
- COMPETITOR** in **Sends** - **FULL** (every competitor must belong to some country)
- COMPETITOR** in **Competes** - **FULL** (a competitor must compete in at least one event)
- EVENT** in **Competes** - **FULL** (no event has no competitors)
- EVENT** in **Holds** - **FULL** (every event must be held at some venue)
- VENUE** in **Holds** - **FULL** (every venue must hold at least one event – or it wouldn't be a venue)





Olympic Games





Olympic Games: 5) Attributes

5) Consider the *Potential Attributes* of each entity type – and their datatypes

COUNTRY

Potential Attributes	Datatype
country_code (Identifier)	char(3)
country_name	varchar(30)
continent	char(10)
number_of_competitors	smallint
team-manager	varchar(30)

Relationships can also have attributes

Competes

Potential Attributes	Datatype
result	char(10)
time	time(7)
medal_won	char(6)

An attribute should NOT be repeated in any other table
the PK/FK match will be considered later

An attribute name suggesting a number doesn't necessarily have to have a numeric datatype
e.g., room number – 16C22

A table should NOT have two or more attributes describing some other potential entity type.
its okay to have the single attribute team_manager in the table about a country.

But, if you also add the Team Manager's Mobile Number then you have two attributes about another entity type (TEAM MANAGER) in a table which should be about COUNTRY only.

Connection traps: fan & chasm



Connection Traps

Connection Traps

Misinterpretation of the meaning of certain relationships can give rise to problems when designing an ER model - **Connection Traps**.

There are two main types of connection trap - **Fan Traps** and **Chasm Traps**.



Fan Trap

Fan Trap

Occurs between related entities that are **not directly connected** and the indirect pathway that connects them includes two **1:N relationships** that **fan out** from a central entity.

This means that certain entity occurrences that are related can only be connected using a pathway that can be **ambiguous**.

Consider to following E-R Diagram:



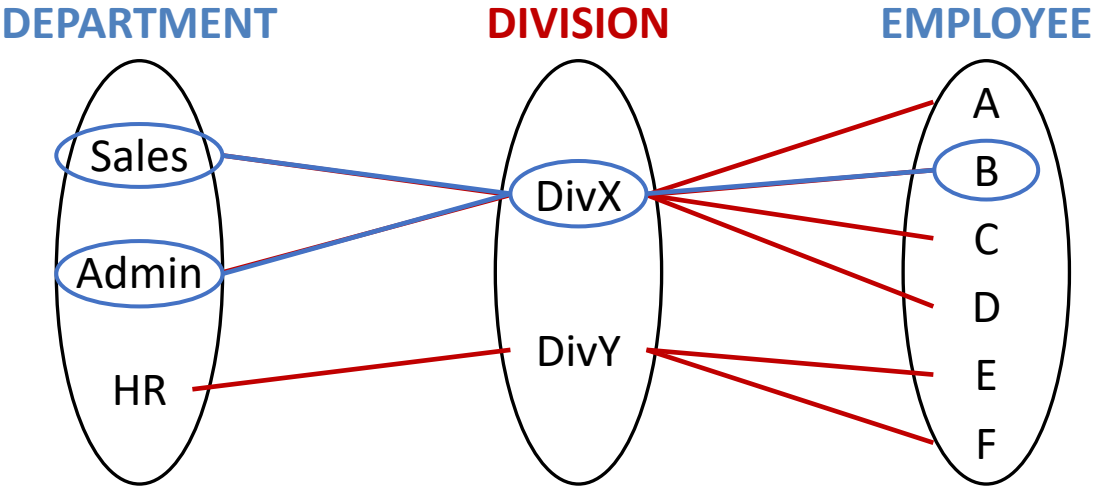
There is an **indirect relationship** between **DEPARTMENT** and **EMPLOYEE** – **employs**

Because the E-R diagram doesn't directly model the **employs** relationship, finding which department a particular employee is employed in **can be ambiguous**.

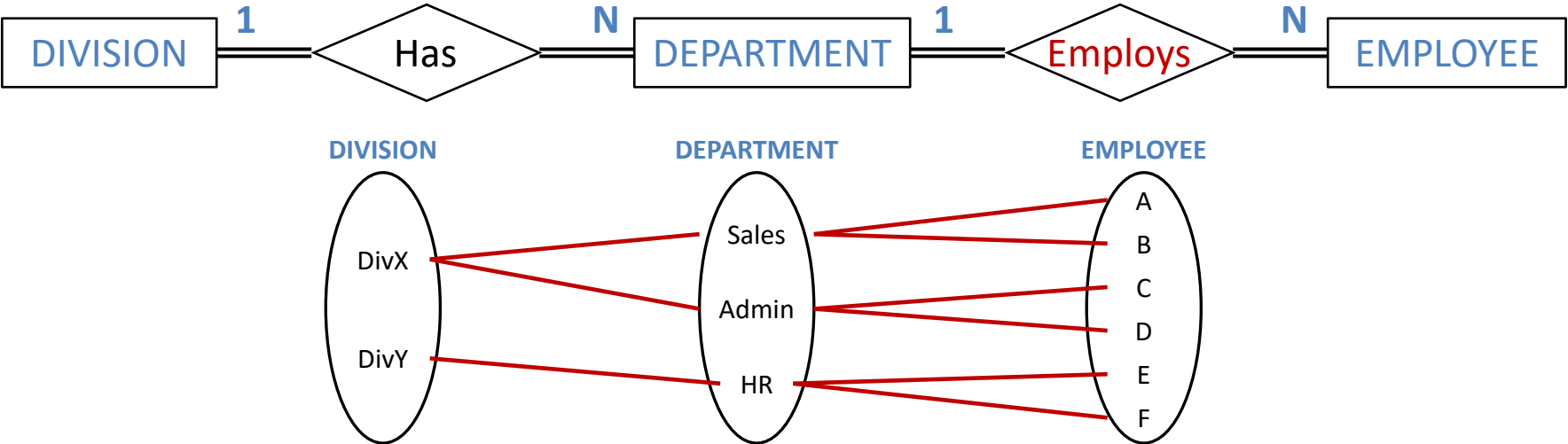


Fan Trap

In which Department is B employed ?
- Ambiguous – Sales or Admin ??



The Fan Trap is **resolved by restructuring the E-R Diagram** to represent the correct association between the entities





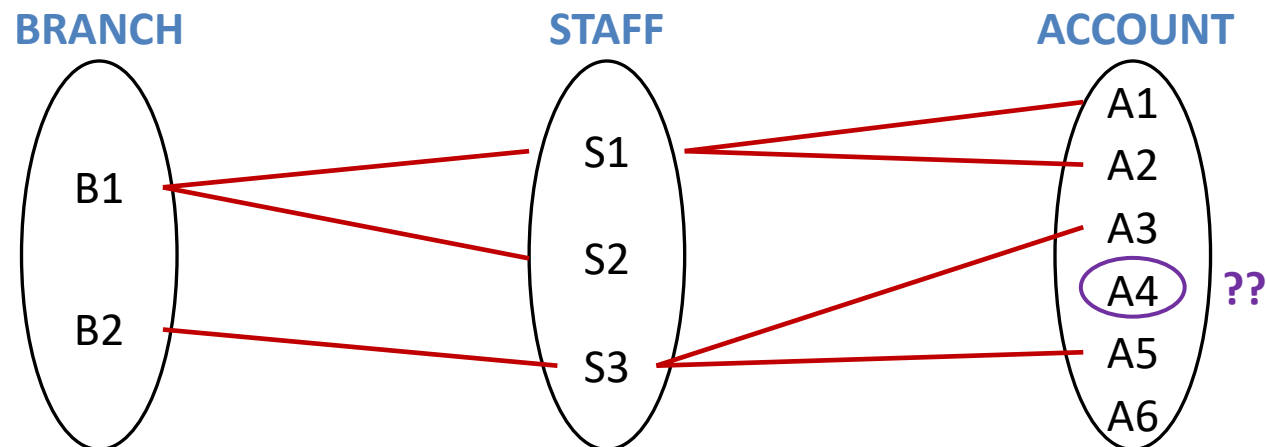
Chasm Trap

Chasm trap

Occurs between **related entities that are not directly connected** and the indirect pathway that connects them includes **partial participation**.

This means that certain entity occurrences that are related have **no means of connection**.

Consider the following E-R Diagram:



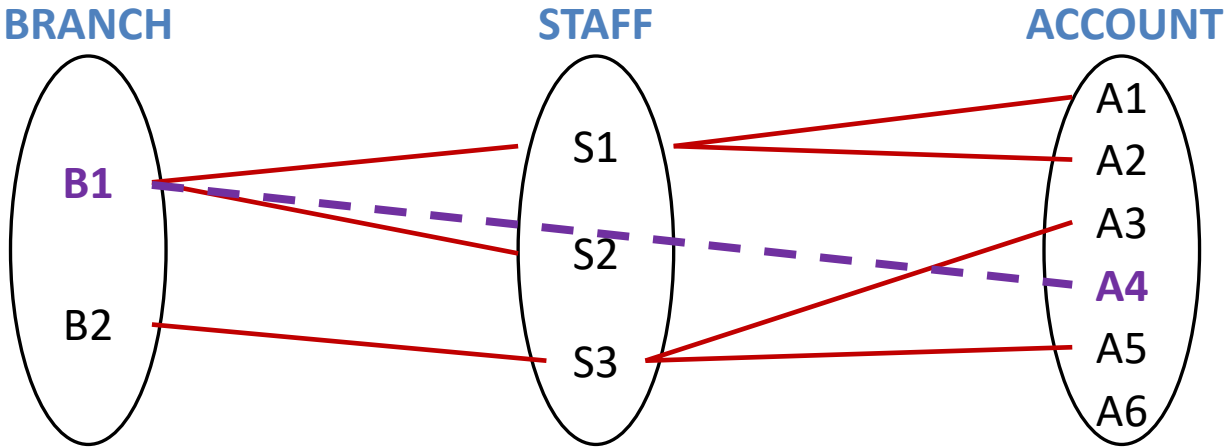
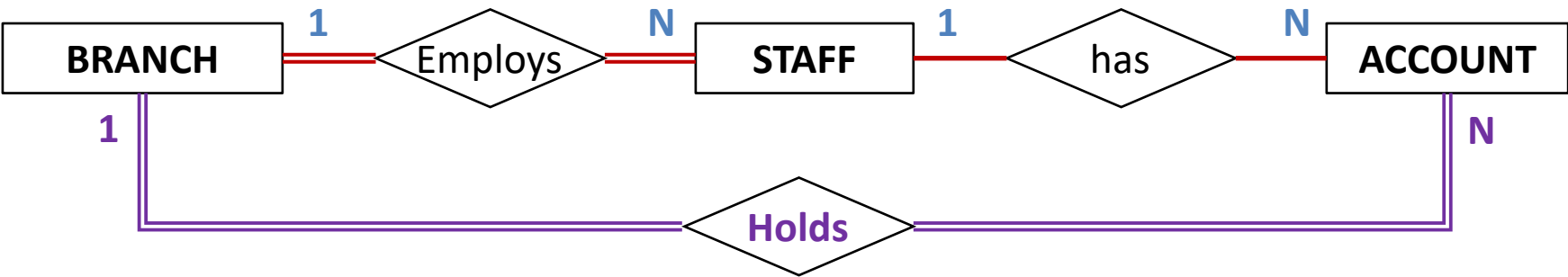
**Which BRANCH
Holds
ACCOUNT A4 ?**



Chasm Trap

Solution?

If it is important that the branch which holds each account is known (it is!) then the **relationship must be modelled** and included in the E-R Diagram.



Adding the **Holds** relationship resolves the chasm trap.

Multiplicities (min, max) notation



E-R Diagram Notations

E-R Modelling is used to capture a **high-level conceptual data model** of a business scenario prior to implementation of a database.

The E-R Diagram shows the basic data structures (in terms of the **entities**, their **attributes** and the **relationships** between them) and the main business **constraints**.

Cardinality ratios and participation constraints are together referred to as **multiplicities** and are often shown on the E-R Diagram using a **(min, max) notation**.



Annotated with multiplicities - (min, max) notation:



Multiplicity records the **minimum** and the **maximum** number of times an instance of an entity is expected to take part in a relationship.

For the **Used_in** relationship:

A **part** might not be used in any project (**min – 0**), but may be used in many projects (**max - ***)

A **project** must use at least one part (**min – 1**), but may also use many parts (**max - ***)



E-R Diagram Notations

The most common multiplicity constraints are:

(0,1) - Zero or one entity occurrence

(1,1) - Exactly one entity occurrence

(0,*) - Zero or many entity occurrences

(1,*) - One or many entity occurrences

If necessary, multiplicity constraints can be specified more accurately.

e.g., (3,8) means **at least 3** occurrences of an entity in a relationship and **at most 8**.

Multiplicities reflect the business rules which will be enforced when the database is implemented – so it's important to get them right!

For example, (from the previous slide) since the multiplicity of **PROJECT** in the **Used_in** relationship is (1,*), *it will not be possible to have a project which does not use a part.*

There are a range of other symbols and annotations used with E-R diagrams to help model real world situations (see Table C.1 in Connolly & Begg (6th ed)).

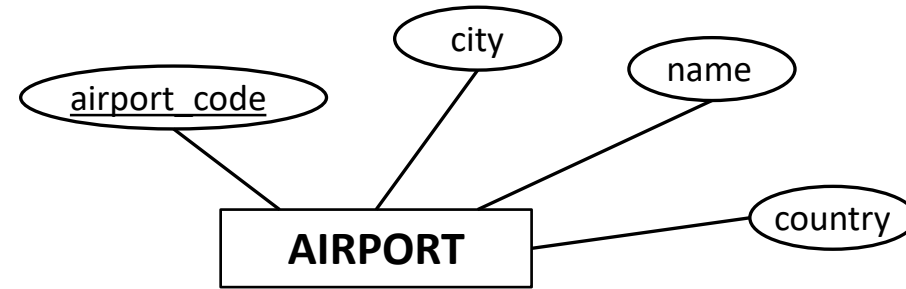
The most common will be introduced here, others will be introduced as and when they are needed.



Attributes

Can become cluttered on a complex diagram.
Instead, list the attributes for each entity separately, with the primary key underlined

AIRPORT (airport_code, city, name, country)



Weak Entities

An entity type that is **existence-dependent** on some other entity type *i.e.*, **only exists if another entity exists**, is called a **weak entity type**

A weak entity type has **no independent existence**.

e.g., children of employees, next-of-kin of patients, etc.

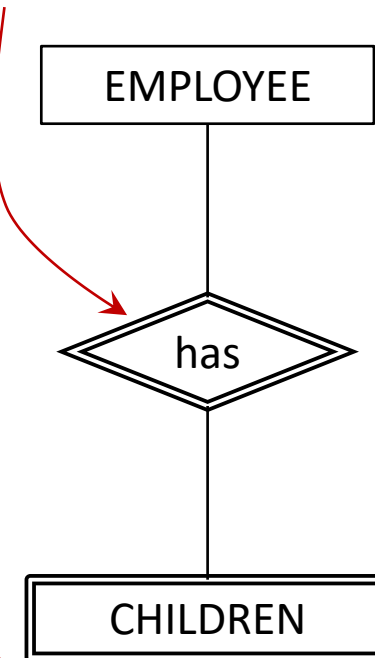
Therefore – **no primary key**; existence is dependant on a **parent entity**.

Consider an employer who (for some reason) keeps a record of the children of his/her employees - **first name and date of birth**.

This is not sufficient to identify the children **unambiguously**.

A child can be **uniquely identified only** in the context of the employee information.

Identifying Relationship



Weak Entity Type



You will find several different E-R modelling conventions and notations used in the literature.

In this module, we use the **Chen** notation

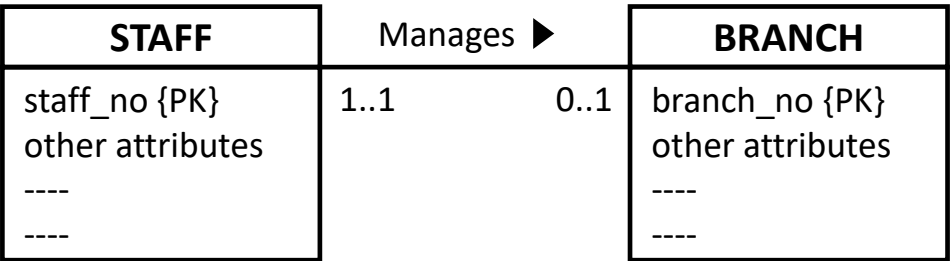
*see Table C.1 in
Connolly & Begg (6th ed)*

Connolly and Begg use a **UML** notation

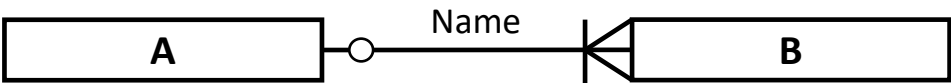
You may also see **Crow's Foot** notation

*UML – Unified Modelling Language
widely used in object oriented design*

UML Example



Crow's Foot Example



One-to-many relationship with
optional participation for entity A
and full participation for entity B



Steps for Creating an E-R Diagram

1. Identify all **Entity Types** with their **attributes** and **identifiers**
Entity Types are normally nouns in the system descriptions
An attribute is a property of an entity type
Identifiers are the potential primary keys
2. Identify all **Relationships** between **Entity Types**
Relationships are normally verbs (or verb clauses) relating two entity types in the system description
Relationships may have attributes
3. Determine the **Cardinality Ratios** of each relationship
1:1, 1:N, N:1 or M:N
4. Identify the **Participation Constraints** of each entity type in each relationship
Full or Partial
Cardinality ratio and participation constraints determine the multiplicity of an entity in a relationship
5. Combine all of the above to produce E-R Diagram

Mapping rules: key copying & link tables



Mapping to the Relational Model

We have been studying the process of designing a database :

- ✓ 1. Creating an **E-R Diagram** based on the results of a data investigation (**E-R Modelling**) ✓
2. Using the mapping (translation rules) to create a **relational schema** (a set of linked tables)
3. Checking that each table is correctly designed (i.e. in **3rd Normal Form**) which may involve decomposition (splitting into more tables)

Steps 1 & 2 are part of **ER modelling**; 3 is **Normalisation**

Mapping to the Relational Model involves :

Creating a **relational table** for each entity type

Linking the tables to reflect the **relationships** identified in the ER Diagram

This is achieved using the **mapping rules** which specify for a particular cardinality ratio (and sometimes a particular **participation constraint**) how the tables should be linked (to allow joins in SQL)



Mapping to the Relational Model

There are three main steps to the mapping process

1. **Create a Relation** (or table) for each Entity Type 
*Entity identifier becomes the **primary key***
2. **Link the Tables** using the **Relational Mapping Rules**
either - *Copy primary keys to other tables as **foreign keys***
or - *Create relationship relations (**linking tables**)*
3. **Complete Relations**
*Remove **derived attributes** (calculated from existing attributes)*
*Add '**active attributes**' to relationship relations*
Remove unnecessary tables

Links (or **common attributes**) are built between entity types to express the relationships identified in the E-R model

We will consider the two basic approaches in turn:

1. *Key Copying*
2. *Link Table Creation*



Mapping to the Relational Model

Key Copying

The Primary Key (**PK**) of one table is copied (or posted) to another table as a Foreign Key (**FK**) to express a relationship

For example, consider **DEPARTMENT** *Employs* **EMPLOYEE (1:N)**

With tables **DEPARTMENT** (dno, dname) and **EMPLOYEE** (eno, ename, salary)

DEPARTMENT

dno	dname
1	Sales
2	Admin

We could post the **PK** from **DEPARTMENT** (**dno**) to **EMPLOYEE**

EMPLOYEE

eno	ename	salary	dno
101	Dopey	21000	1
102	Grumpy	25000	2
103	Happy	19000	2
104	Sneezy	22000	1
105	Sleepy	25000	2

Adding **dno** as a **FK** in **EMPLOYEE** expresses the **1:N** relationship from **DEPARTMENT** to **EMPLOYEE**

This allows **DEPARTMENT** and **EMPLOYEE** to be joined to connect **EMPLOYEE** records with corresponding **DEPARTMENT** records

The tables are now :

** non-standard notation;
used here to denote a FK*

DEPARTMENT (dno, dname) and **EMPLOYEE** (eno, ename, salary, **dno***)



Mapping to the Relational Model

Link Table Creation

For example, consider **SUPPLIER** **Ships** **PART (M:N)**

With Tables: **SUPPLIER** (sno, supp_name) **PART** (pno, part_name)

To add the **Ships** relationship **(M:N)** - create a **link table** called **SHIPS**.

The **PK** of the new **SHIPS** table is a **composite** of those from **SUPPLIER** and **PART**

i.e., **SHIPS**(sno*, pno*, quantity)

SUPPLIER

sno	supp_name
135	Camco
159	J&J Parts

PART

pno	part_name
P13	Flange Plate
P28	Sprocket Cover
P42	Speed Limiter

SHIPS

sno	pno	quantity
135	P13	130
135	P42	57
159	P13	93
159	P28	75
159	P42	10

The link table (**SHIPS**) may also contain '*active attributes*' (normally amounts or dates) such as, e.g., '**quantity**' which belong only in a linking table.

Also, to express the **M:N Ships** relationship, the **SHIPS** link table contains duplicate **sno** values **AND** duplicate **pno** values.

Effectively **SUPPLIER** to **SHIPS** is **1:N** AND **PART** to **SHIPS** is **1:N**



Mapping to the Relational Model

Comparison of Mapping Approaches

Key Copying

does not involve extra tables (no extra join overhead)

not possible for M:N relationships

Link Tables

uses extra linking table (extra join overhead)

necessary for M:N relationship

reduced null values in foreign keys of some 1:1 or 1:N relationships (where certain participation constraints are partial)

Where possible, null values (blank value) should be reduced by design

1:1 Mapping Rule

Copy **PK** of either relation (*but not both!*) to the other as **FK**

For example, consider the relationship **EMPLOYEE Assigned CAR**

EMPLOYEE (enum, ename) **CAR** (regnum, make, model)

Copy primary key in either direction

EMPLOYEE (enum, ename, **regnum***) **CAR** (regnum, make, model)

or **EMPLOYEE** (enum, ename) **CAR** (regnum, make, model, **enum***)



Mapping to the Relational Model

1:1 Mapping rule - Example

A **1:1** relationship is implemented as a **special form of a 1:N** relationship where the **FK MUST contain UNIQUE values**

*In a **1:N** relationship a FK **SHOULD NOT** have unique values*

For example, in **EMPLOYEE** Assigned **CAR** (with *enum* as FK in **CAR**)

EMPLOYEE

enum	ename
101	Tom
103	Dick
104	Harry

CAR

regnum	make	model	enum
GJZ 1498	Ford	Focus	104
EFZ 5391	Renault	Scenic	101
SEZ 7980	Ford	Fiesta	103

Note that *enum* in **CAR** (**FK**) must have **unique** values

Could also have used *regnum* as FK in **EMPLOYEE** – again, must have **unique** values

If (**and only if**) *regnum* is the **only** data to be held about **CAR**, then the relational schema can be simplified by

deciding that **CAR** is not an entity (i.e., should not be modelled) and adding *regnum* (the unique identifier) as an attribute in **EMPLOYEE**



Mapping to the Relational Model

1:N Mapping rule

Copy primary key from the '1' relation to the 'N' relation as **foreign key**

For example, consider the relationship **WARD Contains PATIENT (1:N)**

WARD (wardnum , wname) **PATIENT** (patnum , pname)

Copy PK of the '1' table (**WARD**) to the 'N' table (**PATIENT**) as FK

i.e., **WARD** (wardnum , wname) **PATIENT** (patnum , pname, **wardnum***)

In **PATIENT**, the FK (**wardnum**) should contain duplicate values – i.e. there should be more than one patient in each ward

M:N Mapping Rule

Create a **Link Table** using primary keys from **both** tables as the **composite primary key**

- Normally use the relationship name as the Link Table Name
- PK of Link Table is **composite**
- Other '**active attributes**' may be added if appropriate

For example, consider the relationship **STUDENT Sits EXAM (M:N)**

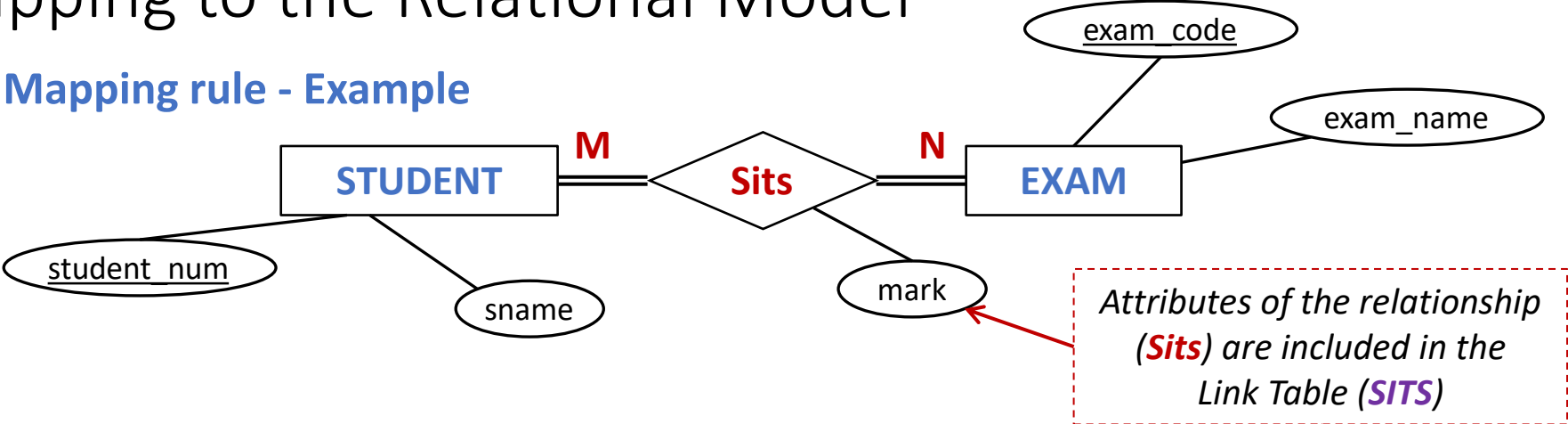
STUDENT (student_num, sname) **EXAM** (exam_code, exam_name)

Link Table **SITS** is created - **SITS** (student_num*, exam_code*, mark)



Mapping to the Relational Model

M:N Mapping rule - Example



STUDENT

student_num	sname
12345	J Goodall
23456	P McCann

Alternatively, could use a unique identifier (**id**) as the **PK** in the Link Table (**SITS**) instead of using a composite key

EXAM

exam_code	exam_name
COM140	Computer Technology
COM106	Database
COM101	Programming

SITS

id	Student_num	exam_code	mark
1	12345	COM140	56
2	12345	COM106	62
3	12345	COM101	48
4	23456	COM140	69
5	23456	COM106	50

Completing the relational schema



Mapping to the Relational Model

When the relational mapping rules have been applied, and the tables are linked, all that remains in constructing the relational model is to **complete the relations**:

1. Remove Derived Attributes

Derived attributes are those which can be **calculated** from other existing attribute(s).

They do not need to be stored (can be calculated in reports, queries etc.)

Examples of Derived Attributes:

*Total Cost = Unit Cost * Number Ordered*

Pension Contribution = %age of Salary

In practice derived attributes are normally omitted earlier in the design process

2. Add 'Active Attributes' to Relationship Relations

Active attributes tend to be **amounts or dates/time**

We have already seen an example when mapping the **Sits** relationship between **STUDENT** and **EXAM**

The 'mark' attribute of the **Sits** relationship is added to the **SITS** linking table

See also the projemp database used in the SQL practicals;

The **WORKS** table has the nonkey attributes 'role' & 'duration' –

WORKS (eno*, pno*, **role**, **duration**)



Mapping to the Relational Model

3. Remove Unnecessary Tables

Sometimes tables contain attributes found in other tables and do not link to more than one other table. It may be possible to **eliminate** such tables thereby reducing the join overhead.

For example, consider the relationship **EMPLOYEE** **Awarded** **QUALIFICATION** (**M:N**)

Applying the mapping rules gives:

EMPLOYEE (eno , ename) **AWARDED** (eno*, **qual***) **QUALIFICATION** (**qual**)

Since attribute **qual** is also found in **AWARDED** –
QUALIFICATION is unnecessary and can be **eliminated**



Mapping to the Relational Model

Summary of the steps in Mapping to the Relational Model

Mapping rules are applied to an E-R model to create a **relational schema** (a set of linked tables)

1. Create a Relation (or table) for each Entity Type

2. Link the Tables using the Relational Mapping Rules

1:1 relationship - Copy PK into other table as FK (in any direction but NOT both)

1:N relationship - Copy PK from the '1' table into the 'N' table as FK

M:N relationship - Create link table (using both PKs); PK on new Link Table is composite

3. Complete the relations

Remove derived attributes

Add 'active attributes' to relationship relations

Remove unnecessary tables

The result is a **Relational Schema** (a set of linked tables), such as , for example, the projemp relational schema used in the labs:

DEPT (dno, dname, location)

EMP (eno, ename, salary, age, supno, dno*)

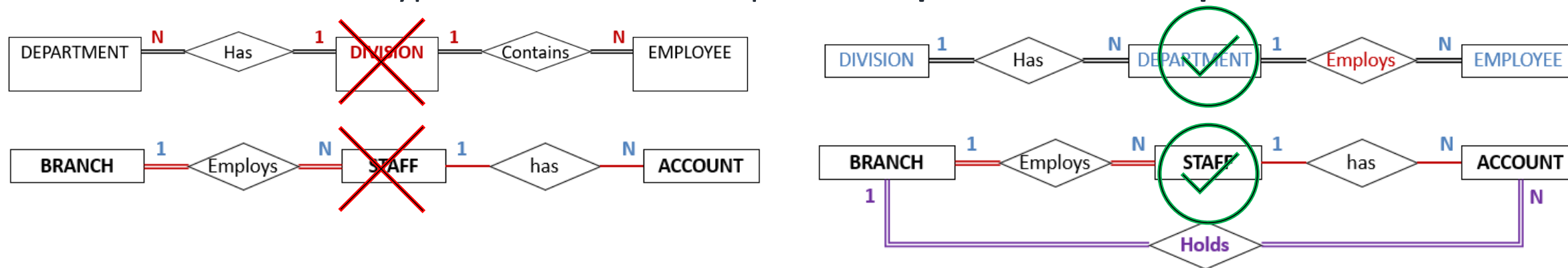
WORKS (eno*, pno*, role)

PROJ (pno, pname, ptype, budget)



Conclusion

- There are two main types of connection trap - **Fan Traps** and **Chasm Traps**.



- Multiplicity records the **min**imum and the **max**imum number of times an instance of an entity is expected to take part in a relationship.



- Link the Tables using the Relational Mapping Rules**

1:1 relationship - Copy PK into other table as FK (in any direction but NOT both)

1:N relationship - Copy PK from the '1' table into the 'N' table as FK

M:N relationship - Create link table (using both PKs); PK on new Link Table is composite



**THE
END**